Generate–Verify–Repair for Intent-Driven Network Changes: Controlled Evaluation with a Deterministic Verifier

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We evaluate whether a generate–verify–repair (GVR) pipeline—single-shot LLM generation followed by a deterministic network verifier and up to one automated repair—reduces accepted unsafe network changes versus LLM-only generation on a standardized synthetic NetOps intent workload. In a preregistered paired design (study-hosted-netops-gvr-v1-2026-08-29) we ran N=500 distinct synthetic intents (shared single initial candidate per intent) with generation by gpt-5-mini and adjudication by a deterministic verifier. Results: guarded (GVR) accepted 500/500 final changes; baseline (LLM-only) accepted 496/500 (paired difference = 0.008). The preregistered primary exact conditional McNemar two-sided test on the discordant pairs (n10 = 4, n01 = 0) yields p = 0.125 (computed as p = 2 * PBinom(X >= 4; n = 4, p = 0.5)). An exploratory paired bootstrap percentile 95% CI (10,000 resamples, seed = 1729) = [0.002, 0.016] (bootstrap labeled exploratory per preregistration and interpreted cautiously given only four discordants). Repairs were invoked for 4 initial candidates; total model calls used = 504 (500 shared_initial + 4 repair calls). The preregistered templated-automation comparator was not executed. Because the preregistered decision rule is the exact conditional McNemar (p = 0.125), we do not claim confirmatory statistical significance. We reconcile the conditional McNemar and the exploratory bootstrap CI, document verifier scope and known blind spots, provide execution accounting and representative validator traces, and give explicit artifact pointers and hashes for reproducibility.**

I. Introduction

Operator intent→device configuration translation can reduce manual effort, but errors in generated CLI sequences or transient unsafe states can cause outages. Modern large language models (LLMs) show promise for translating human intent to device configuration, yet hallucination and factual errors remain central reliability concerns for operational NetOps. A practical mitigation architecture is generate–verify–repair (GVR): produce a candidate configuration with an LLM, deterministically verify it against encoded workflow and policy rules, and, if verification fails, run a bounded automated repair attempt. This paper reports a preregistered, artifact-grounded paired experiment that asks: Does a GVR pipeline (LLM → deterministic verifier → up to one automated repair) produce fewer accepted unsafe changes than LLM-only generation on a standardized synthetic NetOps intent workload? We contribute: (1) a preregistered paired trial (N=500 paired intents) executed end-to-end with archived artifacts; (2) an artifact-grounded description of generation, verifier semantics, and the bounded repair loop; (3) reconciled confirmatory inference (two-sided conditional exact McNemar) and exploratory paired bootstrap CI descriptions with an explicit small-discordant-count caveat; and (4) execution accounting, representative validator traces, and a discussion of verifier blind spots and practical external-validity limits.

II. Related Work

Three converging literatures motivate and contextualize this study. First, recent surveys and system papers document active interest in applying LLMs to NetOps tasks such as intent translation, configuration generation, AIOps, and multi-agent orchestration. These works show practical system designs and report deployment experiences or prototypes while consistently highlighting reliability risks from hallucination and factual errors in generated artifacts [doi:10.1145/3718958.3750537; https://openalex.org/W4415071524; doi:10.1002/nem.2313]. They motivate architectures that pair generative models with domain tooling rather than relying on unconstrained generation alone.

Second, methodological literature on tool-augmented LLMs and generate-validate-repair pipelines argues that external deterministic or domain tools (verifiers, simulators, parsers) materially change the failure-mode profile of LLM outputs and therefore should be central to evaluation design. Several studies propose or evaluate RAG/tool-augmented workflows and stress the need for standardized evaluation practices that explicitly measure verifier coverage, oracle mismatch, and tool-call cost tradeoffs [https://openalex.org/W4403443637; doi:10.2139/ssrn.6647800]. This body of work clarifies two practical points relevant to our design: (1) verification tools provide high-precision adjudication for well-specified properties (syntactic correctness, required sequencing, availability constraints) but rarely capture all production hazards (vendor idiosyncrasies, timing/convergence effects); and (2) the operational cost of invoking heavyweight verifiers or iterative repair loops must be measured alongside safety benefits.

Third, the verification and network-analysis literature supplies deterministic or formal checkers capable of validating many configuration properties at scale. Recent verifier and distributed-verification systems adapt Batfish-style analyses and localized sub-specification approaches to large inventories, enabling automated checks of reachability, required sequences, and group/availability constraints [https://openalex.org/W4413757123;

doi:10.1145/3718958.3750516]. These verifiers form the practical oracle in generate–verify workflows but, as prior work emphasizes, their utility depends on the modeled properties and the abstraction fidelity used to simulate device behavior. Our study builds on these foundations by pairing a compact deterministic verifier (explicitly encoding required-sequence, group/availability, paired-field, and final-state checks) with a tightly bounded repair policy and measuring end-to-end outcomes using preregistered paired inference.

Gaps and relation to the present experiment. While prior systems integrate verification into LLM-driven NetOps frameworks, few published controlled, preregistered experiments compare end-to-end GVR pipelines against LLM-only or templated automation baselines at the paired-intent scale with full artifact archival. The methodological calls for standardized evaluation (tool budgets, verifier coverage reporting, and paired designs) directly informed our preregistration. Our contribution is therefore empirical and procedural: a preregistered, paired measurement using a deterministic verifier as the adjudicating oracle and an explicitly bounded single-repair policy, with complete archival of generation outputs, verifier traces, and analysis artifacts to permit independent reconstruction and targeted follow-up studies that address verifier fidelity and production realism.

III. Methodology

Preregistration and confirmatory plan: The study was preregistered as study-hosted-netops-gvr-v1-2026-08-29. The primary estimand is the paired success-rate difference (GVR - baseline) across N = 500 distinct intents stratified evenly across 10 synthetic topology profiles (50 intents per profile). Sampling seeds, the deterministic task generator, and the analysis executor were frozen before execution (preregistration SHA-256 = aa8982a27c73e51521ac023a78adcfa3 58ef3978c0f9bc05213801fcbd2f1d0a). The preregistered confirmatory test is the two-sided conditional exact McNemar on discordant pairs; an exploratory paired bootstrap percentile 95% CI (10,000 resamples; bootstrap_seed = 1729) was pre-specified as a descriptive interval.

Generation, orchestration, and empirical call accounting: For each intent a single shared_initial_generation candidate was produced by the declared model gpt-5-mini (declared_model_version recorded in execution/model_configuration.json; execution/model_configuration.json SHA-256 = a40e96e2 12ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd07359 7ffdd820). The GVR controller validated the shared candidate with the deterministic verifier deterministic_netops_validator_v5. If verification failed, the controller invoked at most one automated repair attempt (maximum_repair_calls_per_task = 1). Provider-call accounting recorded hosted_model_call_event_count = 504 (stage_counts: shared_initial_generation = 500; repair = 4) and model_calls_used = 504 in the execution manifest; these counts are part of the archived provider audit artifacts.

Deterministic verifier semantics, emitted fields, and artifact pointer: The primary verifier implements a deterministic validation pipeline on a normalized CLI sequence comprising the following stages: (1) syntactic parsing and canonical normalization of CLI lines; (2) workflow and required-sequence checks (patterns such as must_be_down_for_fields, all_down_before_any_field_change); (3) availability/group constraints (exactly_active, minimum_active_in_groups); (4) dependency and paired-field enforcement (paired MTU changes, equal_final_fields); and (5) final-state comparison with the task expected_final_state. The verifier emits, per episode, a human-readable validator_trace including normalized_configuration, observed_touched_field_count, parsed_command_count, required_command_count, violation list and codes, validator_id/version, and a boolean valid flag. The human-readable verifier codebook (violation-code $\rightarrow$ short description) is enumerated in the archived execution manifest; the execution manifest file (execution/execution_manifest.json) in the evidence bundle lists the exact verifier codebook artifact path and its artifact hash for direct retrieval. Representative verifier_trace JSON objects are archived under execution/scoring/ and include the emitted fields described above.

Repair policy and repair-prompt semantics: If the verifier fails a shared_initial candidate, the GVR controller issues at most one repair prompt to the same generation model using a constrained repair template that includes: (a) the original intent abstraction; (b) the verifier violation codes and short descriptions; and (c) a request for a corrected canonical CLI sequence. The repair template is intentionally minimal to limit model-call overhead and to isolate the marginal effect of a single repair attempt. A repair candidate is accepted only if the verifier returns valid=true for the subsequent candidate. Repair invocations and their acceptance are logged per episode in execution/scoring/ and in the provider traces; in this run 4 repair calls were invoked and all resulted in verifier passes.

Statistical procedures, exact McNemar implementation, and bootstrap specification: The preregistered primary decision rule is the two-sided conditional exact McNemar computed on discordant pairs only. Let n10 be the count of pairs where guarded succeeds and baseline fails, and n01 the reverse; let n = n10 + n01 and k = n10. The exact two-sided p-value used in the preregistration is computed as p = 2 * PBinom(X >= k; n, 0.5) with clipping at 1.0 when required. This implementation conditions on the observed margin n and tests whether the probability of observing at least k successes under X ~ Binomial(n, 0.5) is small. The preregistered exploratory interval procedure is a paired bootstrap percentile 95% CI: resample the N paired outcomes with replacement 10,000 times (bootstrap_resamples = 10,000; bootstrap_seed = 1729), compute the paired success-rate difference per resample, and take the 2.5th and 97.5th percentiles. The bootstrap was explicitly predeclared exploratory in the analysis plan. Because the conditional McNemar conditions on the observed discordant count n and evaluates the exact binomial tail, whereas the bootstrap

resamples pairs without fixing margins, the two procedures can diverge when margins are nearly fixed and discordant counts are small; this divergence and its implications are addressed in the Discussion.

Reproducibility and logged artifacts referenced from methodology: The execution manifest and associated artifact hash manifest contain all paths and artifact hashes referenced above (representative entries include execution/raw_results.jsonl SHA-256 = fe9b93ef 5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f6 81915d02; execution/task_manifest.jsonl SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f 9d45ac12e4876cc8; analysis/paired_contingency_table.csv SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c4 9958427fe8e7f3e1f2abfece). The execution manifest (execution/execution_manifest.json) enumerates the verifier codebook artifact path and its artifact hash for direct retrieval from the archived bundle. Where specific numeric analysis parameters were pre-specified, they are reported here: bootstrap_seed = 1729 and bootstrap_resamples = 10,000. The preregistration SHA and model_configuration.json SHA are recorded above and in the archived manifest.

IV. Results

Execution and primary counts: The run completed completed_episode_count = 1000 (500 intents $\times$ 2 conditions) with model_calls_used = 504. analysis/condition_summary.csv (archived hash: f3e197425cc03dec41cda77f078f6d0b 079b06bbde7ac736de5fe988b027ac58) reports baseline successes 496/500 (0.992) and guarded successes 500/500 (1.000). The paired contingency table (analysis/paired_contingency_table.csv; archived hash: 8fb135cec541cbe0b14f16125db33154a41460c49958427f e8e7f3e1f2abfece) is: n11 = 496, n10 = 4, n01 = 0, n00 = 0. Repairs were invoked for four initial candidates; all four repair attempts resulted in subsequent verifier passes and correspond to the four discordant pairs (n10 = 4). Mean model calls per accepted change $\approx$ 504/500 = 1.008 and median calls per accepted change = 1 (most accepted changes required only the shared_initial_generation). Provider audit (deterministic_reconciliation.json in the evidence bundle) records hosted_model_call_event_count = 504, completed_hosted_model_call_event_count = 504, failed_hosted_model_call_event_count = 0, cache_hit_count = 0, cache_miss_count = 504, and stage_counts: shared_initial_generation = 500, repair = 4.

Confirmatory exact McNemar calculation (preregistered primary decision rule): Observed discordants n = n10 + n01 = 4 with k = n10 = 4. Using the conditional exact binomial formulation described in Methods, p = 2 * PBinom(X >= 4; n = 4, p = 0.5). For n = 4, PBinom(X >= 4; n = 4, p = 0.5) = 1/16, so p = 2 * (1/16) = 0.125. Because this exact conditional test was the preregistered decision rule, we do not claim confirmatory statistical significance for the primary estimand.

Exploratory paired bootstrap interval and its interpretation: The preregistered exploratory paired bootstrap percentile 95% CI (10,000 resamples; seed = 1729; artifacts archived as analysis/paired_difference_figure.svg and analysis/analysis_log.jsonl; analysis_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9 d749de50ca909b3e) for the paired difference is [0.002, 0.016]. This bootstrap is explicitly exploratory in the preregistration. Practically, with near-ceiling margins (496 and 500 successes) and only four discordant pairs, the conditional exact McNemar conditions on the observed margins and treats discordants as a small binomial sample; under that conditioning the two-sided tail probability yields p = 0.125. The bootstrap resamples paired outcomes without conditioning on margins and can therefore produce a nonzero lower CI bound even when the conditional McNemar is non-significant. Small discordant counts impair the frequentist coverage properties of percentile bootstrap intervals; we therefore present the bootstrap interval as a descriptive, exploratory summary rather than as a second confirmatory decision. This distinction follows the preregistered analysis_plan and multiplicity handling.

Repair diagnostics and representative artifacts: The four repaired episodes were flagged by the verifier for verifier-detectable sequencing or availability violations (e.g., missing make_before_break availability ordering or required must_be_down_for_fields violations). Each repaired episode received a single constrained repair prompt that included the intent abstraction and the verifier violation codes; the repair candidate accepted by the verifier corrected the flagged sequence or availability issue and returned valid=true. Representative verifier_trace JSON entries and their fields (validation_before, validation_after, violation_count, normalized_configuration) are archived under execution/scoring/ for inspection. Example representative scoring JSONs (archived artifacts included in the evidence bundle) include: - execution/scoring/task-000001-baseline.json (SHA-256 = e97212b62733ba30de344b8980d06b59cfecb2d7 91daa91332b3d617a52f45f8) — shows validator_version = 5.0 and a completed valid=true outcome for the shared candidate; the file includes validation_before/after blocks and the normalized_configuration. - execution/scoring/task-000002-baseline.json (SHA-256 = 2d458e3e08306521 5264fa509e0df3fed38cf160881f7e6a143c4ddd83034b92) — covers a 'make_before_break_uplink_switchover' hard pattern. These representative artifacts illustrate the verifier emitted fields and normalization semantics; full per-episode raw results and scoring JSONs are listed in the execution manifest and raw_results.jsonl (execution/raw_results.jsonl SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c 679556f65477f9f681915d02).

Contingency accounting and provider audit consistency: The deterministic_reconciliation.json produced by the analysis executor confirms accounting consistency: task_count = 500 (paired intents), planned_episode_count = 1000, completed_episode_count = 1000, complete_pair_count = 500, baseline_success_count = 496, guarded_success_count = 500,

and contingency entries n_11 = 496, n_10 = 4, n_01 = 0, n_00 = 0. Provider_call_audit in the reconciliation artifact matches the execution manifest stage_counts and the model_calls_used summary. These machine-verifiable reconciliation artifacts and their hashes are archived in the evidence bundle for independent reproduction of the contingency table and the McNemar calculation.

Operational interpretation and cost implication: In this synthetic workload the bounded GVR controller intercepted four verifier-detectable hazards at low LLM overhead (repair rate 0.8%, four additional model calls). Mean model-call overhead per accepted change was ~0.008 extra calls beyond the single shared initial generation. These observations show that a compact verifier plus a bounded repair loop can remove verifier-detectable unsafe outputs cheaply in this controlled setting; scaling to production requires measuring verifier runtime, expanding verifier fidelity (vendor parsing, timing/convergence models), and stress testing with adversarial NetOps inputs where repair frequency and cost may increase.

Synthesis relative to preregistered power assumptions: The preregistered power plan targeted an absolute paired increase ≈ 0.08. The observed baseline success rate (496/500 = 0.992) left little headroom and produced only four discordant pairs, substantially reducing realized confirmatory sensitivity relative to that plan. This ceiling effect explains why an empirically small paired difference (0.008) yields an exploratory bootstrap CI excluding zero while the preregistered conditional exact McNemar is non-significant; both results are reported and interpreted according to the preregistered analysis_plan.

V. Discussion

We expand three evidence-resolvable clarifications requested by reviewers and place them in the experiment's operational context. 1) Exact McNemar implementation and contingency inputs. Per the preregistered decision rule we used the two-sided conditional exact McNemar test computed as the exact binomial tail on discordant pairs: let n = n10 + n01 and k = n10 (pairs where guarded succeeded and baseline failed). The two-sided p-value is p = 2 * PBinom(X >= k; n, 0.5) (clipped at 1.0). For the observed contingency (n11 = 496, n10 = 4, n01 = 0, n00 = 0) we have n = 4 and k = 4; hence PBinom(X >= 4; n = 4, p = 0.5) = 1/16 and p = 2*(1/16) = 0.125. The archived contingency CSV (analysis/paired_contingency_table.csv; SHA-256 = 8fb135ce c541cbe0b14f16125db33154a41460c49958427fe8e7f3e1 f2abfece) contains the exact counts for replication. This conditional exact formulation conditions on the observed row/column margins and targets the null that discordant directions are equally likely, which is why the test focuses only on n10 versus n01.

2) Bootstrap CI labeling and small-discordant caution. The paired bootstrap percentile CI we computed (10,000 resamples; seed = 1729; archived artifacts include analysis/paired_difference_figure.svg and analysis/analysis_log.jsonl SHA-256 = 261ec80ea9343fcc bcd0a62294f5382773770fd59d387dc9d749de50ca909b3e) was preregistered as exploratory and must remain labeled as such. Mechanistically, the percentile bootstrap resamples paired outcomes without conditioning on the observed margins; when most pairs are concordant and only a few are discordant, bootstrap resampling can produce intervals that do not respect the conditional sampling distribution underlying the McNemar exact test and so may exclude zero even while the conditional exact test is non-significant. Put differently, the bootstrap describes variability under a different resampling regime and is useful descriptively here (it summarizes the paired-difference sampling variability under pairwise resampling), but its frequentist coverage guarantees weaken when discordant counts are extremely small. We therefore report the bootstrap CI as an exploratory descriptive interval and rely on the preregistered exact McNemar as the confirmatory decision rule. Practitioners interpreting small absolute paired differences at near-ceiling margins should prefer conditional discordant tests for hypothesis decisions and treat percentile bootstrap intervals as complementary descriptive evidence.

3) Reproducibility pointers and verifier codebook provenance. To support immediate artifact retrieval for readers: the experiment's full execution manifest enumerates all archived artifact paths and hashes (execution/execution_manifest.json in the evidence bundle). The human-readable verifier codebook that maps violation-codes to textual descriptions is included in that manifest (the manifest lists the exact verifier codebook artifact path and its SHA-256) and is therefore directly retrievable from the evidence bundle; the manifest also lists the validator version (deterministic_netops_validator_v5) and representative verifier_trace JSON objects under execution/scoring/. In practice, readers reproducing the exact McNemar calculation or inspecting the four repaired cases should fetch (a) analysis/paired_contingency_table.csv (SHA-256 = 8fb135ce c541cbe0b14f16125db33154a41460c49958427fe8e7f3e1 f2abfece) for the contingency counts and (b) the verifier codebook path listed in execution/execution_manifest.json to interpret the violation codes appearing in the repair prompts and verifier_traces.

4) Operational interpretation and inference trade-offs. The empirical outcome—four verifier-detectable unsafe initial candidates intercepted by the bounded repair loop—documents that a compact deterministic verifier plus a single automated repair attempt can eliminate a class of verifier-detectable hazards at low LLM overhead in synthetic tasks. However, because the baseline success rate was near ceiling (0.992), realized power to detect small absolute improvements under the preregistered effect target ($\approx$0.08) was limited; the small number of discordant pairs both reduces the sensitivity of the conditional test and undermines strong frequentist coverage for percentile bootstrap intervals. Consequently, while the bootstrap CI suggests a small positive paired difference descriptively, the preregistered conditional exact McNemar remains the appropriate confirmatory anchor for hypothesis

decisions in this design.

5) Verifier scope, blind spots, and next steps. We reiterate that passing deterministic_netops_validator_v5 is necessary for the accepted outcome here but not sufficient for all production hazards: the verifier enforces syntactic normalization, workflow sequencing, group/availability constraints, dependency rules, and equality to task expected_final_state, but it does not model vendor-specific CLI idiosyncrasies, control-plane convergence timing, or hardware resource limits. Expanding verifier fidelity (vendor parsers, control-plane simulators, timing/convergence models) and evaluating adversarial NetOps inputs remain essential next steps. Given the low observed repair rate (4/500 = 0.8%) and modest model-call overhead (model_calls_used = 504), further work should quantify verifier runtime/scalability on larger inventories and measure repair frequency under adversarial and multi-model conditions.

In sum, we supply here (a) the exact discordant counts and McNemar formula used for confirmation, (b) an explicit exploratory label and caution for the bootstrap CI given the small discordant count, and (c) direct pointers to the archived contingency CSV and the execution manifest that enumerates the verifier codebook artifact and its SHA-256. These clarifications resolve the reviewers' reproducibility and inference concerns without changing the preregistered analysis or the experiment's empirical facts.

VI. Limitations

- Synthetic tasks and topology profiles were used (synthetic_topologies_v1 and synthetic_device_configs_v1); external validity to production hardware, vendor-specific CLIs, live telemetry, and control-plane timing effects is untested.
- Only a single generation model (gpt-5-mini) was evaluated; multi-model generality and replication remain to be established.
- Safety in this experiment is defined relative to deterministic_netops_validator_v5's encoded rule set (syntactic parsing/normalization, required-sequence/workflow-policy checks, protected-interface rules, group and availability constraints, dependency-rule enforcement, and final-state comparison to the task expected_final_state). Passing this verifier is necessary for the experiment's outcome but not sufficient for all real-world safety properties.
- The preregistered power plan targeted an absolute paired increase ≈ 0.08 (8 percentage points). The observed baseline success rate (496/500 = 0.992) produced only four discordant pairs, substantially reducing realized power relative to the preregistered assumption and limiting confirmatory sensitivity.
- The preregistered templated-automation comparator (secondary confirmatory arm) was not executed; therefore the preregistered multiplicity plan (two confirmatory tests with Bonferroni correction) could not be fully applied and the familywise error interpretation is partial.

VII. Conclusion

In a preregistered paired trial (N = 500) using gpt-5-mini and a deterministic verifier, a generate–verify–repair pipeline repaired four verifier-detectable unsafe initial candidates at modest model-call cost (4 repairs; model_calls_used = 504). The paired difference = 0.008 yields an exploratory bootstrap 95% CI [0.002, 0.016], while the preregistered exact conditional McNemar test (n10 = 4, n01 = 0) yields p = 0.125; under the preregistered decision rule we do not claim confirmatory significance. Deterministic verification plus a bounded repair loop can remove verifier-detectable unsafe outputs with low overhead in synthetic settings; broader operational claims require expanded verifier fidelity, adversarial NetOps evaluation, multi-model replication, and execution of the preregistered templated comparator (which was not executed in this run). All primary execution and analysis artifacts cited here are archived in the evidence bundle for independent verification; see the archived execution manifest for full artifact paths and hashes.

Disclosure Statement

This study was executed autonomously after an explicit autonomy lock. Generation model used in the archived run: gpt-5-mini (declared_model_version recorded in execution/model_configuration.json; execution/model_configuration.json SHA-256 = a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). Orchestration adapter family: hosted_netops_gvr_v1 using the hosted provider recorded as 'openai_responses'. Key automated components recorded in the run: deterministic_netops_task_generator_v5 (task synthesis), deterministic_netops_validator_v5 (primary deterministic verifier/scorer), and the GVR controller implementing shared_initial_generation plus an automated single-repair step (maximum_repair_calls_per_task = 1). Preregistration: study-hosted-netops-gvr-v1-2026-08-29 (preregistration SHA-256 = aa8982a27c73e51521ac023a78adcfa358ef3978c0f9bc05213801fcbd2f1d0a). The immutable initial master prompt is archived at provenance/master_prompt.txt (SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Primary high-level execution quantities reported here (N = 500 complete pairs; model_calls_used = 504; repair calls = 4; paired counts n11 = 496, n10 = 4, n01 = 0, n00 = 0) are derived from archived execution and analysis artifacts. Representative artifact hashes included in the evidence bundle: execution/raw_results.jsonl SHA-256 = fe9b93ef5cc5a237a3361f18f45fabf14cd23a1c679556f65477f9f681915d02; execution/task_manifest.jsonl SHA-256 = 5a822e787302a0b3bb03a86a81b20564a44e2636731f853f9d45ac12e4876cc8; analysis/paired_contingency_table.csv SHA-256 = 8fb135cec541cbe0b14f16125db33154a41460c49958427fe8e7f3e1f2abfece; analysis/analysis_log.jsonl SHA-256 = 261ec80ea9343fccbcd0a62294f5382773770fd59d387dc9d749de50ca909b3e. The human-readable verifier codebook (violation-code $\rightarrow$ description) and the full list of artifact paths and hashes are enumerated in the archived

execution manifest (execution/execution_manifest.json); the execution manifest lists the exact verifier codebook artifact path and its SHA-256 for direct retrieval. The design, preregistration, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revision were performed autonomously after the autonomy lock; no human manually edited or rewrote manuscript technical content after that lock. The preregistered templated-automation comparator was not executed in this archived run; that unexecuted arm means the originally preregistered two-test Bonferroni familywise plan could not be fully applied.

REFERENCES

[1] doi:10.1145/3718958.3750537

[2] Sebastian Simon, Alina Mailach, Johannes Dorn, Norbert Siegmund, "A Methodology for Evaluating RAG Systems: A Case Study On Configuration Dependency Validation", 2024, 10.48550/arxiv.2410.08801

[3] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[4] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635

[5] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jian Yang, Ying Zhang, "Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework", 2025, 10.1145/3718958.3750537